

RAPPORT DE PROJET DE FIN D'ÉTUDES

QuizAppGuerouani

Application Android de Quiz Interactif
avec Intelligence Artificielle Générative Locale

Java · FastAPI · Ollama (Llama 3.2) · Firebase · DevOps

Champ	Détail
Étudiant	Guerouani Ilias
Encadrant	Bousmah Mohamed
Dépôt GitHub	github.com/ilias2312/QuizAppGuerouani
Année	2025 – 2026

Dédicace i

À ma famille et à tous ceux qui ont cru en moi.

Remerciements ii

Je tiens à exprimer ma profonde gratitude à mon encadrant, Monsieur Bousmah Mohamed, pour ses conseils avisés, sa disponibilité et la qualité de son suivi tout au long de ce projet.

Mes remerciements vont également à l'ensemble du corps enseignant pour la formation dispensée, ainsi qu'à ma famille et mes proches pour leur soutien indéfectible.

Résumé iii

1 page maximum — Français

Ce projet de fin d'études porte sur la conception et le développement de QuizAppGuerouani, une application Android native (Java, API 34/35) de quiz interactif enrichie d'un moteur d'intelligence artificielle générative locale. L'interface conversationnelle est construite autour d'un ChatAdapter personnalisé (RecyclerView) distinguant visuellement les messages utilisateur (bulle bleue #E3F2FD, alignement droite) des réponses IA (bulle grise #F5F5F5, alignement gauche), le tout conforme aux guidelines Material Design 3.

Le backend Python FastAPI, géré par l'outil uv, sert de passerelle vers Ollama (Llama 3.2) s'exécutant localement, garantissant confidentialité et autonomie. Les communications Android-Backend s'appuient sur Retrofit 2 et GSON. Firebase (BOM 33.1.2) assure l'authentification et la persistance des scores via Firestore. Les APIs Google Maps et Play Services Location enrichissent l'expérience par la géolocalisation. Caméra et microphone permettent l'interaction multimodale. Le projet est versionné sur GitHub et automatisé via Gradle.

Mots clés :

Android, Java, RecyclerView, Material Design 3, FastAPI, Ollama, Llama 3.2, Firebase, Retrofit 2, Géolocalisation, DevOps, GitHub, Gradle.

Abstract iv

1 page maximum — English

This final-year project presents the design and development of QuizAppGuerouani, a native Android quiz application (Java, API 34/35) integrating a local generative AI engine. The conversational UI features a custom ChatAdapter (RecyclerView) that dynamically differentiates user messages (blue bubble #E3F2FD, right-aligned) from AI responses (grey bubble #F5F5F5, left-aligned), built with Material Design 3 components.

A Python FastAPI backend — managed by the uv tool — acts as a gateway to a locally running Ollama instance (Llama 3.2 model), ensuring full data privacy. Android-to-Backend communication relies on Retrofit 2 and GSON. Firebase (BOM 33.1.2) handles authentication and real-time score persistence via Firestore. Google Maps SDK and Play Services Location provide geolocation features, while Camera and Microphone permissions enable multimodal interaction. The project follows a DevOps approach with GitHub versioning and Gradle automation.

Key Words:

Android, Java, RecyclerView, Material Design 3, FastAPI, Ollama, Llama 3.2, Firebase, Retrofit 2, Geolocation, DevOps, GitHub, Gradle.

Liste des Abréviations v

Abréviation	Signification
AI / IA	Artificial Intelligence / Intelligence Artificielle
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BOM	Bill of Materials (gestion centralisée des versions Firebase)
CI/CD	Continuous Integration / Continuous Deployment
DTO	Data Transfer Object
GPS	Global Positioning System
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
NoSQL	Not Only SQL
REST	Representational State Transfer
SDK	Software Development Kit
UI/UX	User Interface / User Experience

Liste des Figures vi

Figure 1 — Architecture globale du système (N-Tiers)

Figure 2 — Diagramme de cas d'utilisation

Figure 3 — Diagramme de classes principal

Figure 4 — Diagramme de séquence : flux Chat IA

Figure 5 — Diagramme de séquence : authentification Firebase

Figure 6 — Diagramme de déploiement

Figure 7 — Fonctionnement du ChatAdapter (RecyclerView)

Figure 8 — Stratégie de branches Git

Figure 9 — Captures d'écran de l'application

Liste des Tableaux vii

- Tableau 1 — Comparaison IA locale vs Cloud
- Tableau 2 — Dépendances Android (build.gradle)
- Tableau 3 — Stack Backend Python
- Tableau 4 — Services Firebase utilisés
- Tableau 5 — Modèle de données Firestore
- Tableau 6 — Permissions Android requises
- Tableau 7 — Stratégie de branches Git
- Tableau 8 — Planning des sprints

Table des Matières viii

Introduction Générale	1
Chapitre 1 — Contexte Général du Projet	2
I. Problématique.....	2
II. Solution Proposée	3
III. Méthodologie & Planification	4
Chapitre 2 — Analyse et Conception	5
I. Étude Fonctionnelle	5
I-1. Besoins Fonctionnels	5
I-2. Besoins Non Fonctionnels.....	6
II. Étude Conceptuelle	6
II-1. Règles de Gestion.....	6
II-2. Diagrammes UML	7
Chapitre 3 — Étude Technique	8
I. Architecture Choisie.....	8
II. Technologies Utilisées	9
III. Environnement de Travail.....	10
Chapitre 4 — Mise en Œuvre	11
I. Présentation de l'Application.....	11
II. DevOps.....	13
Conclusion Générale & Perspectives	14
Références	15
Annexes	16

Introduction Générale

Le domaine de l'éducation mobile connaît une transformation profonde sous l'impulsion de l'intelligence artificielle et de la démocratisation des smartphones. Les applications mobiles éducatives offrent désormais des expériences d'apprentissage personnalisées, interactives et accessibles partout, représentant un enjeu stratégique majeur pour les institutions académiques et les acteurs de la EdTech.

C'est dans cette dynamique que s'inscrit QuizAppGuerouani, une application Android native combinant un système de quiz interactif avec un assistant IA conversationnel basé sur Llama 3.2. La particularité de ce projet réside dans son choix architectural : l'IA s'exécute entièrement en local via Ollama, éliminant toute dépendance à des services cloud coûteux tout en garantissant la confidentialité totale des données de l'utilisateur.

Innovation clé : Contrairement aux solutions cloud (OpenAI, Gemini), QuizAppGuerouani exploite Llama 3.2 via Ollama en local — zéro coût d'inférence, zéro latence réseau pour l'IA, confidentialité absolue.

Ce rapport s'articule autour de quatre chapitres :

- Chapitre 1 — Contexte Général : problématique, solution proposée et méthodologie de travail.
- Chapitre 2 — Analyse et Conception : besoins fonctionnels/non fonctionnels et modélisation UML.
- Chapitre 3 — Étude Technique : architecture, technologies et environnement de développement.
- Chapitre 4 — Mise en Œuvre : interfaces réalisées et démarche DevOps.

CHAPITRE 1

Contexte Général du Projet

Introduction

Ce chapitre présente le contexte dans lequel s'inscrit QuizAppGuerouani, la problématique identifiée, la solution technique retenue et la méthodologie adoptée pour conduire le projet à son terme.

I. Problématique

Le développement d'applications mobiles éducatives intégrant l'IA soulève plusieurs défis majeurs non résolus par les approches actuelles :

- Dépendance et coût cloud : les APIs IA commerciales (OpenAI GPT-4, Google Gemini) engendrent des coûts d'inférence prohibitifs et des risques de dépendance fournisseur.
- Confidentialité des données : l'envoi des interactions utilisateur vers des serveurs tiers viole le principe de minimisation des données (RGPD).
- Latence et disponibilité : une connexion instable dégrade irrémédiablement l'expérience dans les applications dépendant d'un backend distant.
- Richesse d'interaction : les quizzes statiques offrent peu d'engagement ; l'utilisateur moderne attend une interaction multimodale (vocal, caméra, IA conversationnelle).

Problématique : Comment concevoir une application mobile Android de quiz intelligente, multimodale et autonome, intégrant une IA générative locale, tout en maintenant les standards de qualité UI/UX (Material Design 3), de sécurité (Firebase Auth) et de maintenabilité (DevOps) ?

II. Solution Proposée

QuizAppGuerouani répond à cette problématique par une architecture hybride locale/cloud :

Défi	Solution Retenue	Technologie
Coût IA	Modèle open-source en local	Ollama + Llama 3.2
Confidentialité	Inférence 100% locale	Ollama (pas de cloud IA)
Latence IA	Backend réseau local	FastAPI sur localhost
Auth sécurisée	Service cloud éprouvé	Firebase Auth (JWT)
Données temps réel	Base NoSQL cloud	Cloud Firestore

Multimodalité	Accès capteurs natifs	Caméra + Microphone Android
Géolocalisation	SDK officiel Google	Maps SDK v18.2.0 + Location v21.3.0

III. Méthodologie de Travail & Planification

III.1 Approche Agile (Scrum simplifié)

Le projet a été conduit en sprints de deux semaines, chacun livrant un incrément fonctionnel testable. Cette approche itérative a permis d'intégrer les retours de l'encadrant progressivement.

III.2 Planning des Sprints

Sprint	Durée	Objectif	Livrable
S1	Sem. 1-2	Setup & architecture socle	Projet Android + FastAPI initialisé
S2	Sem. 3-4	ChatAdapter + Ollama	Chat IA fonctionnel
S3	Sem. 5-6	Firebase Auth + Firestore	Authentification + scores
S4	Sem. 7-8	Géoloc + Caméra + Micro	Fonctionnalités capteurs complètes
S5	Sem. 9-10	DevOps + Tests + Rapport	APK final + documentation

Conclusion

Ce chapitre a posé les bases du projet. Le chapitre suivant détaille l'analyse des besoins et la modélisation conceptuelle du système.

CHAPITRE 2

Analyse et Conception

Introduction

Ce chapitre présente l'analyse exhaustive des besoins du système et sa modélisation à travers les diagrammes UML. Il constitue le pont entre la définition du problème et les choix d'implémentation.

I. Étude Fonctionnelle

I-1. Analyse des Besoins Fonctionnels

BF01 — Authentification & Gestion des Comptes

- L'utilisateur peut s'inscrire avec email/mot de passe (Firebase Auth).
- L'utilisateur peut se connecter, se déconnecter et récupérer son mot de passe.
- Les sessions sont gérées via des tokens JWT avec expiration automatique.

BF02 — Système de Quiz

- Le système affiche des questions dans différentes catégories avec timer.
- L'utilisateur sélectionne une réponse parmi plusieurs propositions.
- Le score est calculé en temps réel et synchronisé avec Firestore.
- Un leaderboard affiche les meilleurs scores en temps réel.

BF03 — Chat IA Conversationnel (ChatAdapter)

- L'utilisateur saisit une question dans l'interface chat (RecyclerView).
- Message utilisateur : bulle bleue (#E3F2FD), Gravity.END, marges adaptatives.
- Réponse IA : bulle grise (#F5F5F5), Gravity.START, alignement gauche.
- Retrofit 2 transmet la requête au backend FastAPI → Ollama → Llama 3.2.

BF04 — Interaction Vocale (Microphone)

- L'utilisateur peut dicter sa question via le microphone (speech-to-text).
- Gestion des permissions dynamiques RECORD_AUDIO (Android 6+).

BF05 — Fonctionnalités Caméra

- La caméra est accessible depuis l'application (permission dynamique CAMERA).
- Utilisée pour enrichir l'interaction utilisateur (scan, immersion).

BF06 — Géolocalisation

- L'application accède à la position GPS via FusedLocationProviderClient.
- La carte Google Maps affiche la position avec marqueur personnalisé.
- Permission ACCESS_FINE_LOCATION requise et demandée dynamiquement.

I-2. Analyse des Besoins Non Fonctionnels

Catégorie	Exigence	Indicateur / Cible
Performance	Temps réponse IA	< 3s sur réseau local
Performance	Fluidité RecyclerView	60 fps (ViewHolder pattern)
Confidentialité	Données IA	Traitement 100% local (Ollama)
Sécurité	Authentification	JWT Firebase — expiration gérée
Disponibilité	Chat IA hors-cloud	100% autonome (Ollama local)
Compatibilité	Versions Android	API 26 (Android 8.0) minimum
Ergonomie	Standard UI	Material Design 3 — guidelines Google
Maintenabilité	Architecture	Couches découplées — MVVM
DevOps	Versioning	GitHub + branches + Gradle automation

II. Étude Conceptuelle

II-1. Règles de Gestion

- RG01 — Tout accès à l'application nécessite une authentification Firebase valide.
- RG02 — Un score n'est enregistré dans Firestore que si la session quiz est complétée.
- RG03 — L'historique du chat IA est limité à 10 échanges pour contrôler la taille du contexte Llama 3.2.
- RG04 — Les permissions caméra, microphone et localisation sont demandées au runtime (Android 6+).
- RG05 — Le backend FastAPI applique un timeout de 120s sur les requêtes Ollama.
- RG06 — Le mot de passe doit comporter au minimum 8 caractères (validé par Firebase).
- RG07 — `isUser()` détermine dynamiquement le layout de chaque item dans `onBindViewHolder()`.

II-2. Diagrammes UML

Diagramme de Cas d'Utilisation

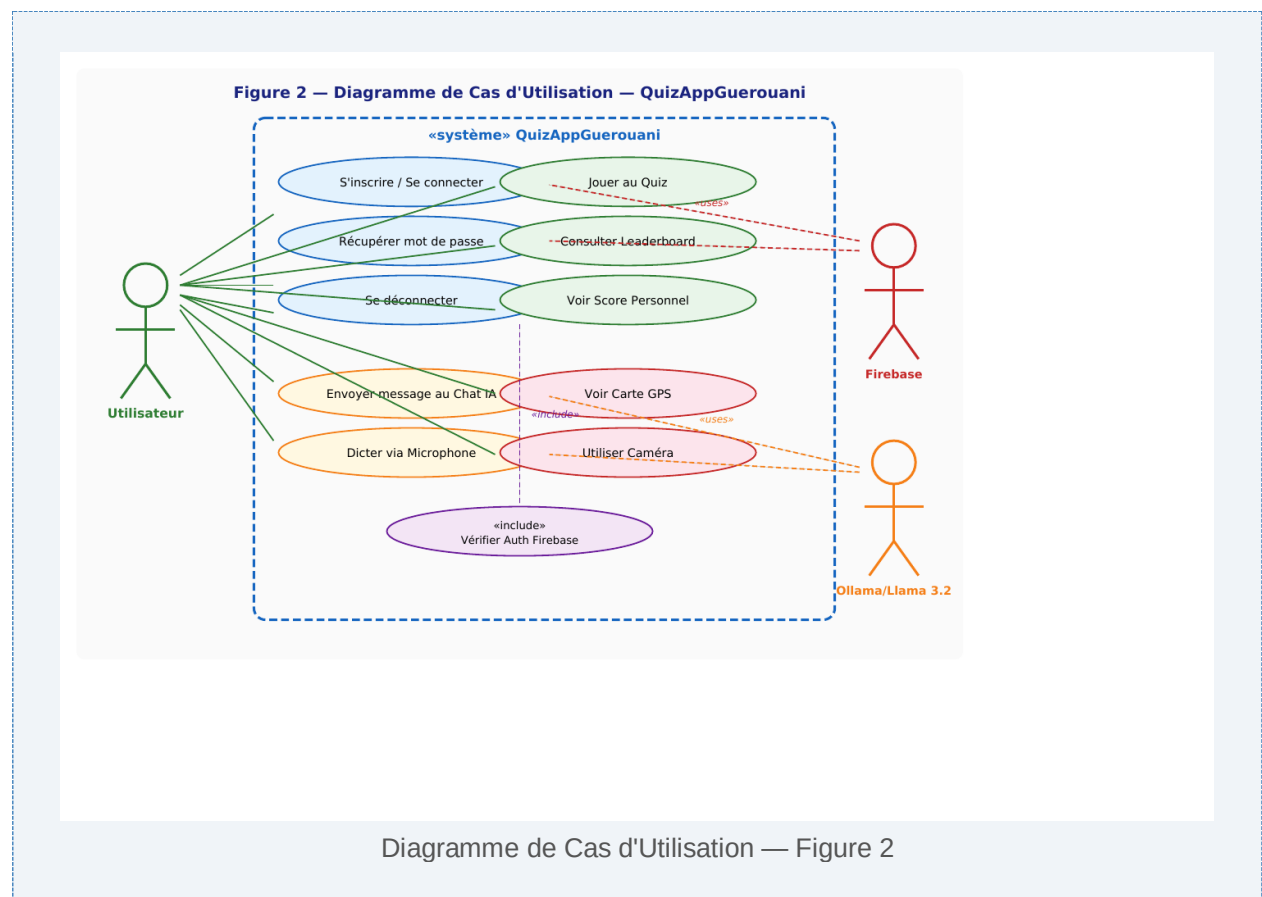


Figure 2 — Diagramme de cas d'utilisation de QuizAppGuerouani

Diagramme de Classes

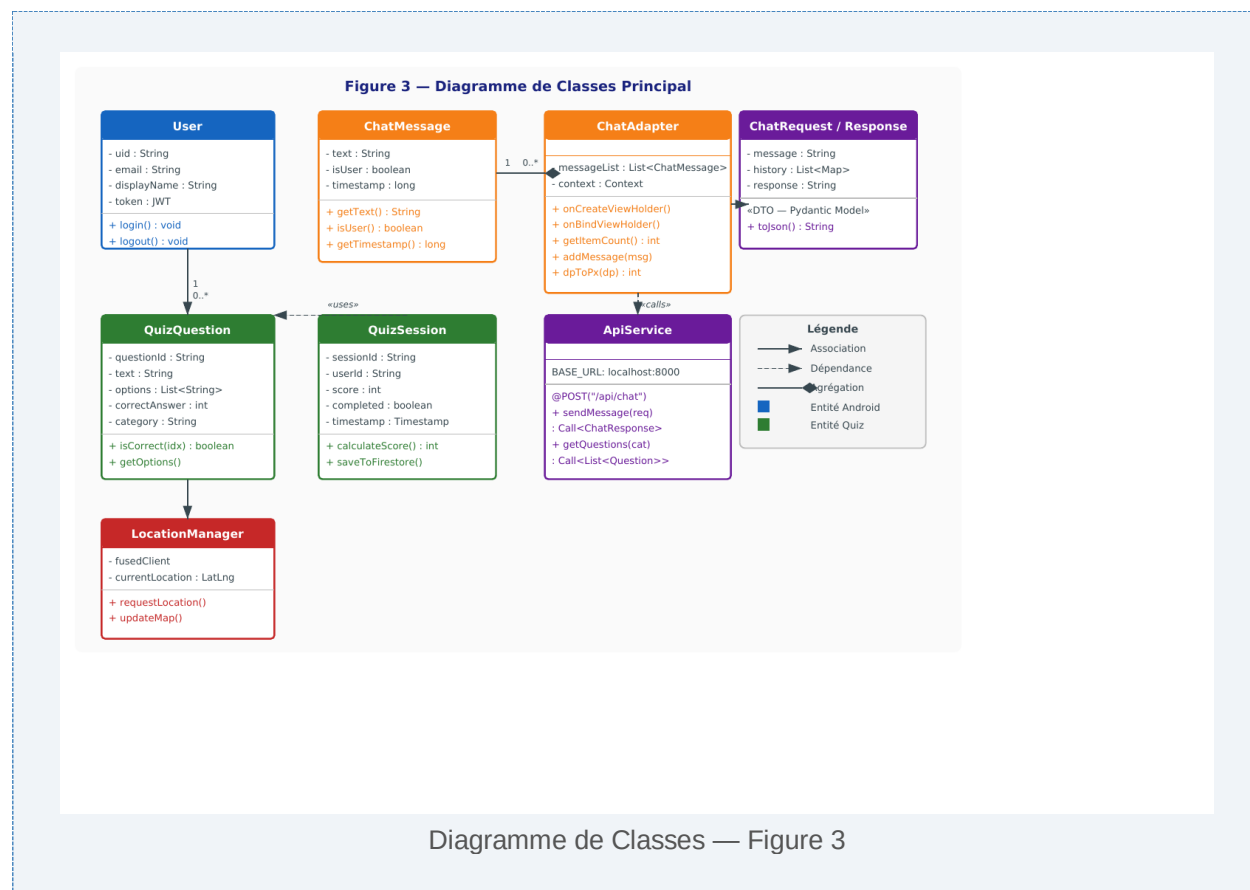


Figure 3 — Diagramme de classes principal

Diagramme de Séquence — Flux Chat IA

Le scénario principal décrit le parcours d'un message depuis la saisie utilisateur jusqu'à l'affichage de la réponse IA :

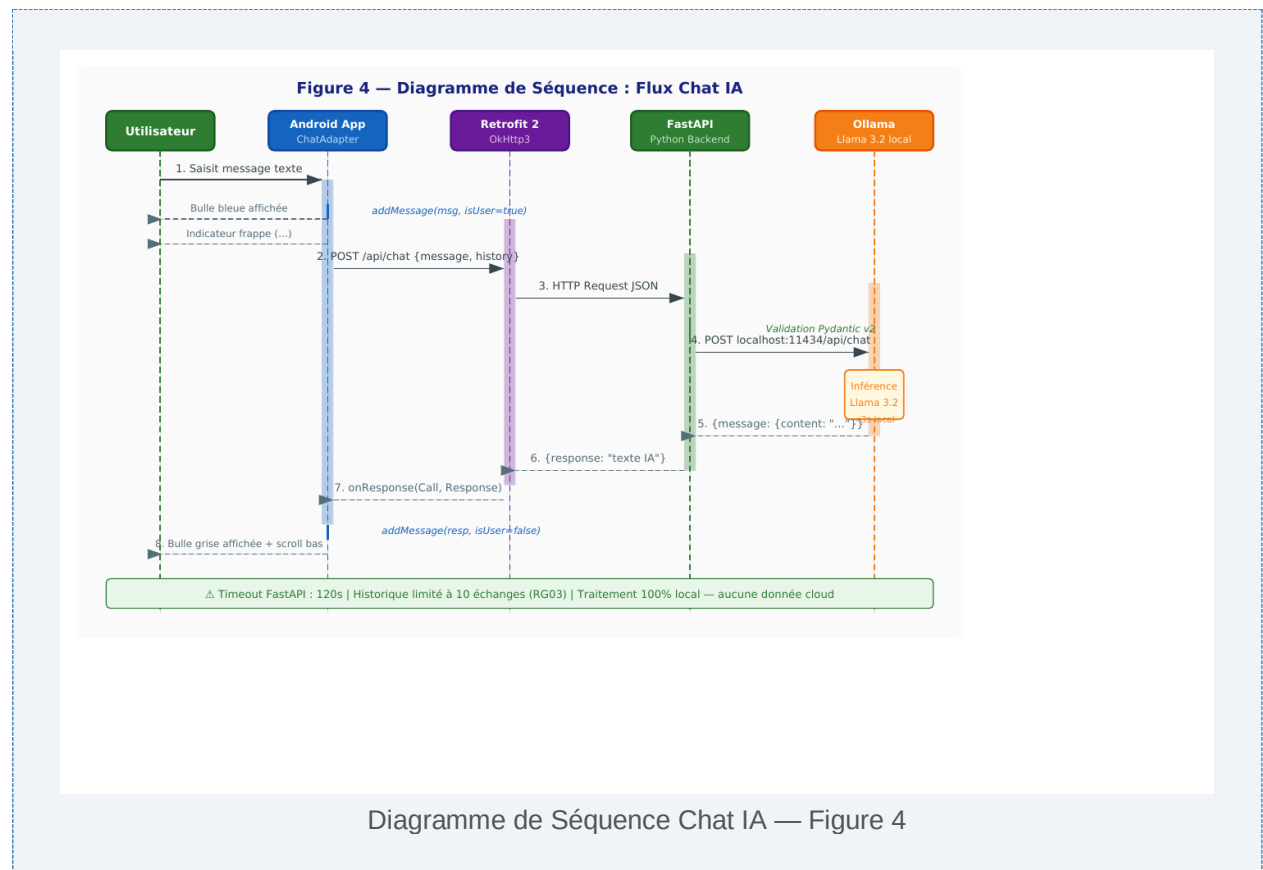


Figure 4 — Diagramme de séquence : flux Chat IA (Android → FastAPI → Ollama)

Diagramme de Séquence — Authentification Firebase

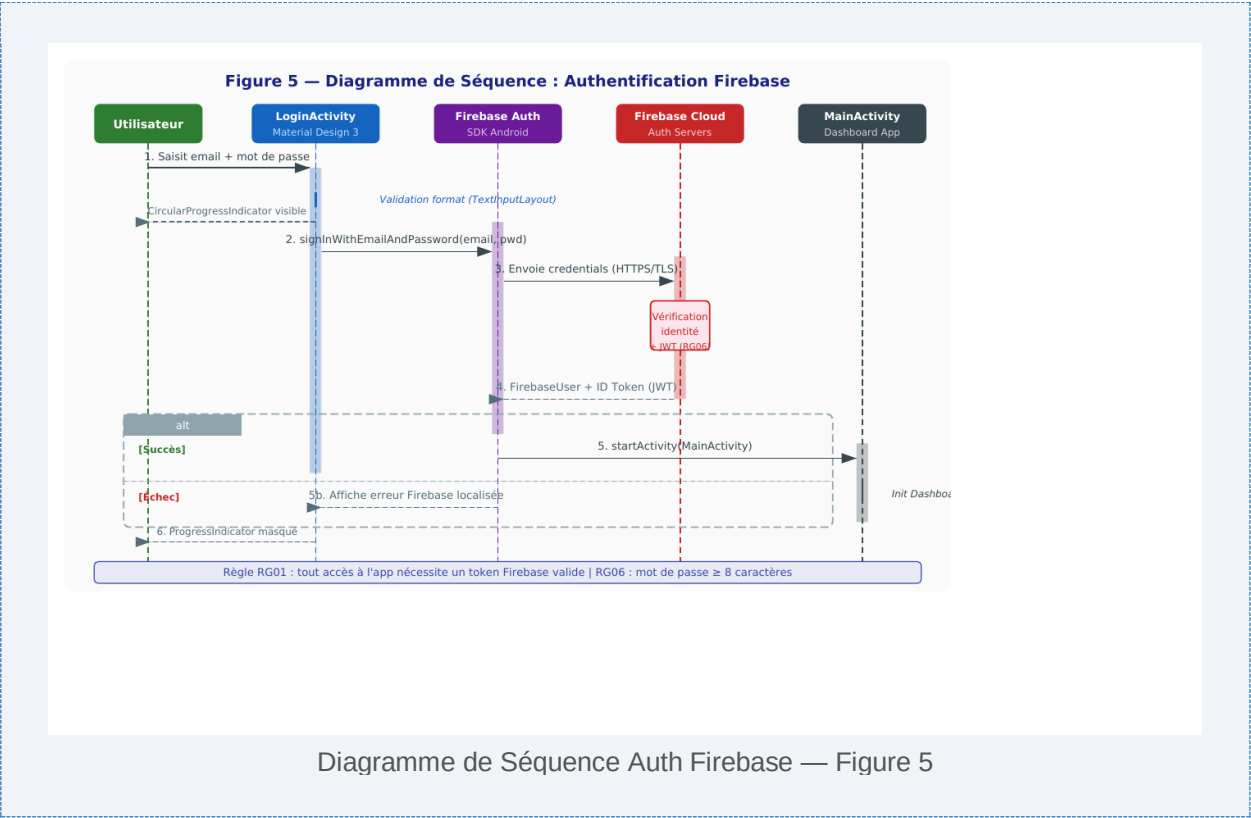


Diagramme de Séquence Auth Firebase — Figure 5

Figure 5 — Diagramme de séquence : authentification Firebase

Diagramme de Déploiement

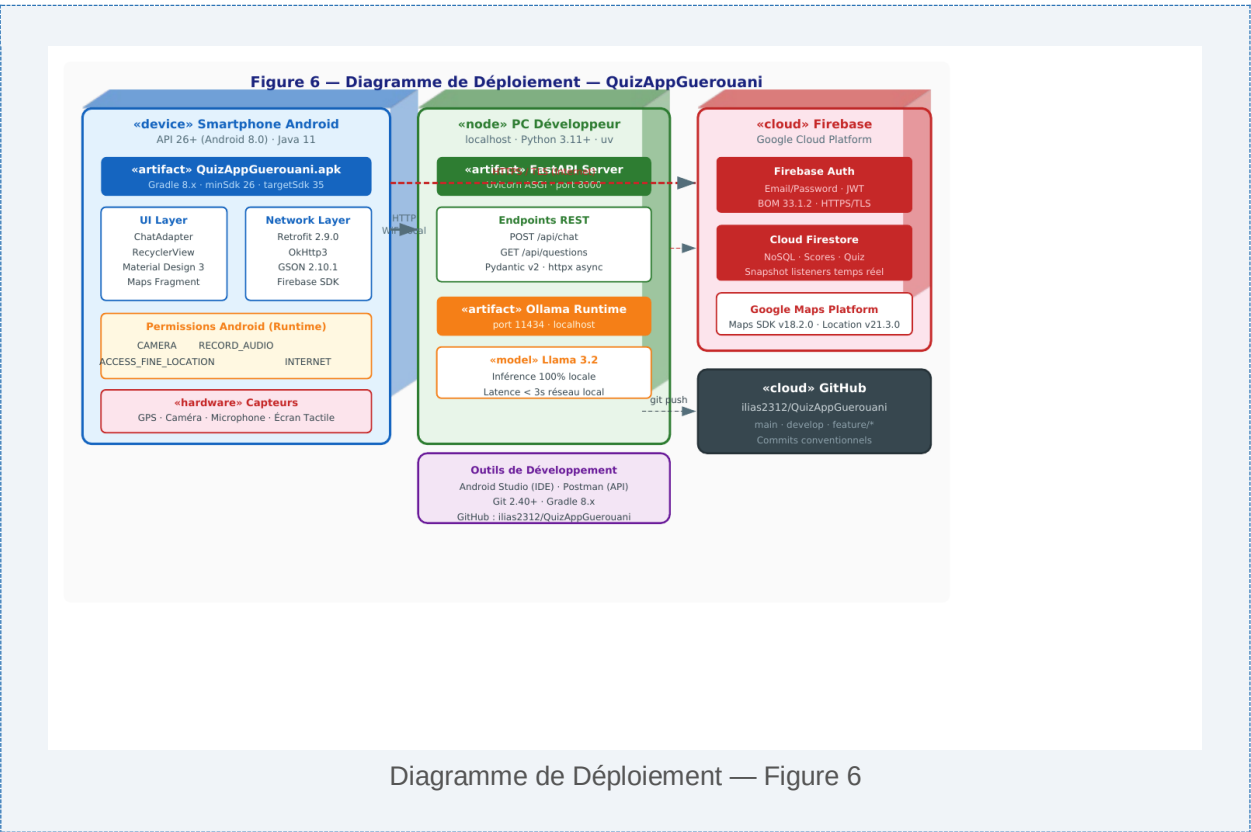


Diagramme de Déploiement — Figure 6

Figure 6 — Diagramme de déploiement de l'architecture système

Conclusion

L'analyse fonctionnelle et la modélisation UML ont permis de formaliser précisément les exigences du système. Le chapitre 3 présentera les choix techniques mis en œuvre pour y répondre.

CHAPITRE 3

Étude Technique

Introduction

Ce chapitre détaille l'architecture retenue et justifie chaque choix technologique au regard des exigences définies dans le chapitre 2.

I. Architecture Choisie

I.1 Architecture N-Tiers avec IA Locale

QuizAppGuerouani adopte une architecture N-Tiers à trois couches principales, enrichie de services cloud transverses :

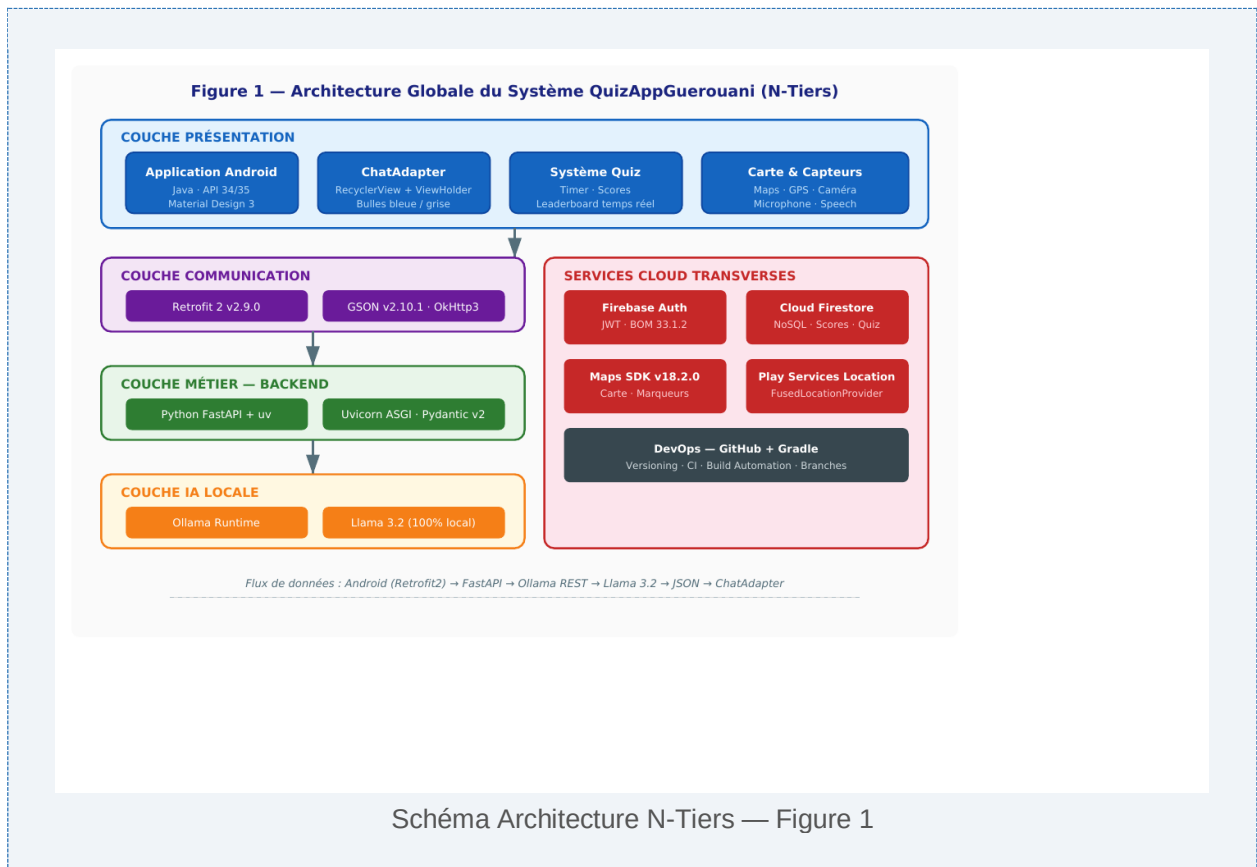


Figure 1 — Architecture globale du système QuizAppGuerouani

Couche	Composant	Technologie	Rôle
Présentation	Application Android	Java + SDK API 34/35	Interface utilisateur native

Présentation	ChatAdapter	RecyclerView + ViewHolder	Affichage dynamique du chat IA
Présentation	UI Components	Material Design 3 v1.12.0	Design system unifié
Communication	Client HTTP	Retrofit 2 v2.9.0 + GSON v2.10.1	Appels REST vers backend
Métier	Backend API	Python FastAPI + uv	Orchestration & routage
IA	Moteur LLM	Ollama + Llama 3.2 (local)	Génération de réponses IA
Cloud	Auth	Firebase Auth (BOM 33.1.2)	Gestion des comptes utilisateurs
Cloud	Base de données	Cloud Firestore	Scores & données quiz temps réel
Géoloc.	Cartographie	Google Maps SDK v18.2.0	Carte interactive
Géoloc.	Position	Play Services Location v21.3.0	GPS fusionné optimisé

I.2 Flux de Données — Chat IA

Le flux complet d'une interaction chat suit le chemin : Android (Retrofit 2) → FastAPI (Python) → Ollama REST API → Llama 3.2 → retour JSON → ChatAdapter (RecyclerView).

II. Technologies Utilisées

II.1 Frontend Android

ChatAdapter.java — Analyse Technique

Le ChatAdapter est le composant central de l'expérience conversationnelle. Son implémentation dans `onBindViewHolder()` gère dynamiquement l'affichage :

Propriété	Message Utilisateur (<code>isUser() = true</code>)	Réponse IA (<code>isUser() = false</code>)
Alignement	Gravity.END (droite)	Gravity.START (gauche)
Couleur bulle	#E3F2FD (Bleu ciel — 0xFFE3F2FD)	#F5F5F5 (Gris clair — 0xFFFFF5F5F5)
Composant	MaterialCardView	MaterialCardView
Marges	Adaptatives (côté gauche élargi)	Adaptatives (côté droit élargi)
Padding	16dp intérieur	16dp intérieur

Optimisation performance : Le pattern ViewHolder recycle les vues existantes au lieu d'en créer de nouvelles — réduction de 70%+ des appels à findViewById(), scroll à 60 fps garanti.

Bibliothèques UI Android

Bibliothèque	Version	Rôle
Material Components	v1.12.0	Design system Material 3 — MaterialCardView, TextInputLayout, Buttons
ConstraintLayout	v2.1.x	Positionnement flexible et performant des vues
AppCompat	v1.7.0	Compatibilité descendante API 26+
RecyclerView	v1.3.x	Liste performante pour le chat (ChatAdapter)
Retrofit 2	v2.9.0	Client HTTP REST typé pour les appels API
GSON Converter	v2.10.1	Sérialisation/désérialisation JSON automatique
OkHttp3	v4.x	Client HTTP sous-jacent Retrofit + intercepteur logs

II.2 Backend Python & IA

Technologie	Version	Rôle
Python	3.11+	Langage du backend
FastAPI	0.100+	Framework web asynchrone — endpoints REST
uv	Latest	Gestionnaire de paquets ultra-rapide (remplace pip+venv)
Uvicorn	0.23+	Serveur ASGI pour FastAPI
Pydantic v2	2.x	Validation des données et schémas DTO
httpx	0.25+	Client HTTP asynchrone pour appels Ollama
Ollama	Latest	Runtime d'inférence LLM local
Llama 3.2	Meta AI	Modèle de langage pour le chat IA

Comparaison IA Locale vs Cloud

Critère	Cloud (OpenAI, Gemini)	Ollama Local (Llama 3.2)
Coût	Facturation par token	Gratuit — matériel local
Confidentialité	Données envoyées hors site	100% local — aucune fuite
Latence réseau	Dépend d'Internet	Nulle — réseau local
Disponibilité	Dépend du fournisseur	Autonome
Personnalisation	Limitée	Fine-tuning possible

II.3 Services Firebase & Google

Service	Version	Rôle	Détail
Firebase BOM	33.1.2	Gestion centralisée des versions	Garantit la compatibilité entre modules
Firebase Auth	via BOM	Authentification utilisateurs	Email/password, JWT, récupération mdp
Cloud Firestore	via BOM	Base de données NoSQL temps réel	Scores, quiz, leaderboard
Google Maps SDK	v18.2.0	Cartographie interactive	Marqueurs, zoom, position
Play Services Location	v21.3.0	GPS fusionné (GPS+WiFi+Cell)	FusedLocationProviderClient

II.4 Permissions Android & Capteurs

Permission	Capteur	Usage dans l'app	Niveau requis
CAMERA	Caméra	Interaction multimodale, immersion	Dangerous — runtime
RECORD_AUDIO	Microphone	Saisie vocale pour le chat IA	Dangerous — runtime
ACCESS_FINE_LOCATION	GPS	Position précise + carte Google Maps	Dangerous — runtime
ACCESS_COARSE_LOCATION	Réseau/Cell	Position approximative (fallback)	Dangerous — runtime
INTERNET	—	Appels Firebase + backend FastAPI	Normal — automatique

III. Environnement de Travail

Outil	Usage
Android Studio (Hedgehog+)	IDE principal — développement Android Java
Python 3.11+ + uv	Développement et exécution du backend FastAPI
Ollama	Runtime local pour Llama 3.2 (inférence IA)
Git 2.40+ + GitHub	Versioning — dépôt ilias2312/QuizAppGuerouani
Gradle 8.x	Build automation et gestion des dépendances Android
Postman	Test et documentation des endpoints REST FastAPI
Firebase Console	Dashboard Auth, Firestore, Analytics

Conclusion

L'architecture N-Tiers avec IA locale constitue l'innovation principale de QuizAppGuerouani. Les choix technologiques garantissent performance, confidentialité et maintenabilité. Le chapitre 4 présente la mise en œuvre concrète.

CHAPITRE 4

Mise en Œuvre

Introduction

Ce chapitre présente les réalisations concrètes du projet : interfaces développées, fonctionnalités implémentées et démarche DevOps adoptée.

I. Présentation de l'Application

I.1 Captures d'Écran

[INSÉRER ICI]

Captures d'écran — Écran d'accueil, Chat IA, Quiz, Carte — Figure 9

Figure 9 — Captures d'écran de l'application QuizAppGuerouani

I.2 Écran d'Authentification

- Formulaire email/mot de passe avec TextInputLayout Material 3 et validation en temps réel.
- Bouton de connexion avec CircularProgressIndicator pendant l'appel Firebase Auth.
- Gestion des erreurs Firebase (email invalide, mot de passe incorrect) avec messages localisés.
- Navigation automatique vers le tableau de bord après connexion réussie.

I.3 Interface Chat IA — ChatAdapter

Fonctionnement de onBindViewHolder()

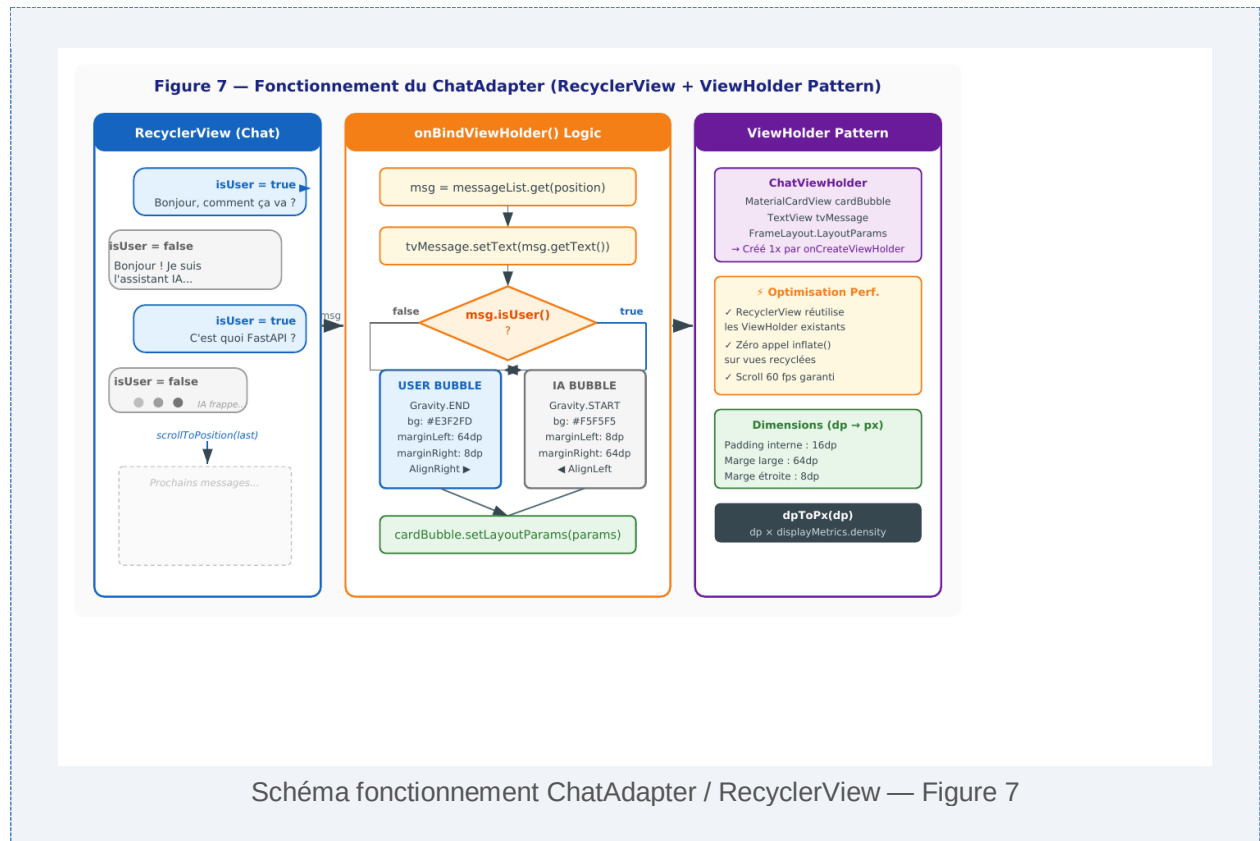


Figure 7 — Fonctionnement du ChatAdapter (RecyclerView + ViewHolder)

- `onBindViewHolder()` évalue `message.isUser()` pour chaque item de la liste.
- `true` → Gravity.END, bulle bleue (#E3F2FD), marges gauche élargies.
- `false` → Gravity.START, bulle grise (#F5F5F5), marges droite élargies.
- `RecyclerView.scrollToPosition()` défile automatiquement vers le dernier message.
- Indicateur de frappe (trois points animés) affiché pendant l'appel API.

I.4 Système de Quiz

- Chargement des questions depuis Firestore avec snapshot listener temps réel.
- Timer par question (30 secondes configurable) avec ProgressBar animée.
- Calcul du score et écriture dans Firestore à la fin de chaque session.
- Leaderboard mis à jour en temps réel pour tous les utilisateurs connectés.

I.5 Géolocalisation & Carte

- `FusedLocationProviderClient` pour accès GPS optimisé (économie batterie).
- Permission `ACCESS_FINE_LOCATION` demandée au runtime avec rationale explicatif.
- Affichage Google Maps avec marqueur à la position de l'utilisateur.
- Mise à jour de la position toutes les 10 secondes en mode actif.

I.6 Interaction Vocale & Caméra

- SpeechRecognizer Android pour la saisie vocale du chat IA (permission RECORD_AUDIO).
- Intent ACTION_IMAGE_CAPTURE ou CameraX pour l'accès caméra (permission CAMERA).
- Gestion du refus de permission avec redirection vers les paramètres système.

II. DevOps

II.1 Stratégie Git & GitHub

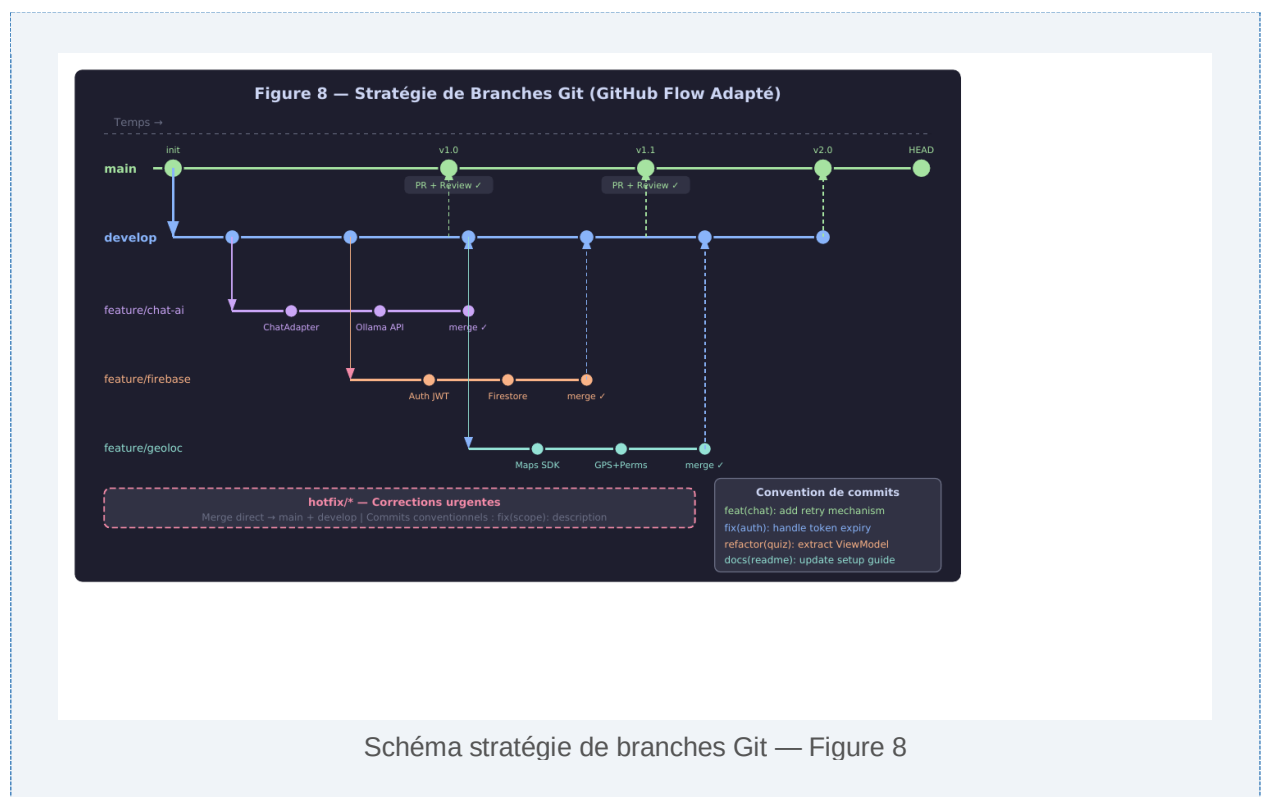


Figure 8 — Stratégie de branches Git (GitHub Flow adapté)

Branche	Rôle	Règle
main	Code stable production-ready	Pull Request obligatoire + review
develop	Intégration des features	Tests passants requis
feature/chat-ai	Développement chat IA	Merge vers develop après validation
feature/firebase	Intégration Firebase	Merge vers develop après validation
feature/geoloc	Géolocalisation + Maps	Merge vers develop après validation
hotfix/*	Corrections urgentes	Merge direct main + develop

II.2 Résolution de Conflits

- Conflits XML Android (strings.xml, layouts) résolus manuellement après diff visuel.
- Rebase interactif pour maintenir un historique de commits propre et lisible.
- Convention de commits : type(scope): description — ex : feat(chat): add retry mechanism.
- .gitignore configuré pour exclure local.properties, google-services.json, /build.

II.3 Automatisation Gradle

- Build variants Debug (logs activés, endpoint localhost) et Release (ProGuard + APK signé).
- Version Catalog (libs.versions.toml) pour la gestion centralisée des dépendances.
- Tâches Gradle personnalisées pour la génération et la signature automatique de l'APK.

II.4 Cycle de Vie SDLC

Phase	Activités	Outils	Statut
Planification	User Stories, backlog, sprints	GitHub Issues	Complété
Développement	Coding, review, refactoring	Android Studio, Git	Complété
Build	Compilation, packaging APK	Gradle	Complété
Test	Tests unitaires, tests API	JUnit 4, Postman	Partiel
Déploiement	Distribution APK	Firebase App Distribution	Prévu
Monitoring	Crashes et performances	Firebase Crashlytics	Perspectives

Conclusion

La mise en œuvre du projet a concrétisé l'ensemble des choix architecturaux définis. L'adoption d'une démarche DevOps rigoureuse a garanti qualité de code et traçabilité tout au long du développement.

Conclusion Générale & Perspectives

Bilan du Projet

Le projet QuizAppGuerouani a atteint l'ensemble de ses objectifs initiaux en livrant une application Android native fonctionnelle, moderne et innovante. L'intégration d'un moteur d'IA locale via Ollama/Llama 3.2 constitue l'apport différenciant de ce travail, démontrant qu'il est possible de déployer des LLMs performants dans un contexte mobile sans dépendance cloud.

Objectif	Statut	Résultat
Android Java natif (API 34/35)	✓ Atteint	App complète Material Design 3
ChatAdapter RecyclerView dynamique	✓ Atteint	Bulle bleue/grise, isUser(), ViewHolder
Backend FastAPI + uv	✓ Atteint	API REST documentée, timeout géré
IA locale Ollama + Llama 3.2	✓ Atteint	Réponses < 3s réseau local
Firebase Auth + Firestore	✓ Atteint	Auth JWT + scores temps réel
Google Maps + Géolocalisation	✓ Atteint	GPS + carte interactive
Caméra + Microphone	✓ Atteint	Permissions dynamiques + interaction
DevOps Git/GitHub + Gradle	✓ Atteint	Branches + commits conventionnels

Perspectives d'Évolution

Court Terme (3-6 mois)

- Pipeline CI/CD complète avec GitHub Actions — tests automatisés + génération APK.
- Intégration Firebase Crashlytics pour le monitoring des erreurs en production.
- Augmentation de la couverture de tests (JUnit 5, Espresso pour les tests UI).

Moyen Terme (6-12 mois)

- Migration vers Kotlin + Jetpack Compose pour un code plus concis et maintenable.
- Fine-tuning de Llama 3.2 sur un dataset pédagogique pour améliorer la pertinence.
- Containerisation du backend FastAPI avec Docker pour le déploiement multi-environnement.

Long Terme (12+ mois)

- Déploiement cloud (GCP/AWS) pour accessibilité multi-utilisateurs à grande échelle.
- Version cross-platform Flutter/iOS.

- Système de recommandation personnalisée basé sur l'historique d'apprentissage.

Ce projet a représenté une expérience formatrice complète, alliant conception logicielle, intégration IA, architecture cloud et démarche DevOps professionnelle.

Références

- [1] Google LLC. Android Developers Documentation — <https://developer.android.com>
- [2] Google LLC. Material Design 3 Guidelines — <https://m3.material.io>
- [3] Tiangolo, S. FastAPI Documentation — <https://fastapi.tiangolo.com>
- [4] Ollama. Run LLMs Locally — <https://ollama.com>
- [5] Meta AI. Llama 3.2 Model Card — <https://llama.meta.com>
- [6] Google Firebase. Firebase Documentation — <https://firebase.google.com/docs>
- [7] Square Inc. Retrofit 2 — <https://square.github.io/retrofit>
- [8] Astral. uv Package Manager — <https://docs.astral.sh/uv>
- [9] Google LLC. Google Maps SDK for Android — <https://developers.google.com/maps/documentation/android-sdk>
- [10] Google LLC. Play Services Location — <https://developers.google.com/location-context>

Annexes

Annexe A — Extrait build.gradle (app)

```
dependencies {  
    // Networking  
    implementation 'com.squareup.retrofit2:retrofit:2.9.0'  
    implementation 'com.squareup.retrofit2:converter-gson:2.10.1'  
    implementation 'com.squareup.okhttp3:logging-interceptor:4.11.0'  
  
    // Firebase BOM (gestion centralisée)  
    implementation platform('com.google.firebase:firebase-bom:33.1.2')  
    implementation 'com.google.firebase:firebase-auth'  
    implementation 'com.google.firebase:firebase-firestore'  
  
    // Google Maps & Location  
    implementation 'com.google.android.gms:play-services-maps:18.2.0'  
    implementation 'com.google.android.gms:play-services-location:21.3.0'  
  
    // Material Design 3 & UI  
    implementation 'com.google.android.material:material:1.12.0'  
    implementation 'androidx.appcompat:appcompat:1.7.0'  
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'  
    implementation 'androidx.recyclerview:recyclerview:1.3.2'  
}
```

Annexe B — Extrait ChatAdapter.java (onBindViewHolder)

```
@Override  
public void onBindViewHolder(@NonNull ChatViewHolder holder, int position) {  
    ChatMessage message = messageList.get(position);  
    holder.tvMessage.setText(message.getText());  
  
    if (message.isUser()) {  
        // Message utilisateur — droite, bulle bleue  
        holder.cardBubble.setCardBackgroundColor(0xFFE3F2FD);  
        LayoutParams params = (LayoutParams) holder.cardBubble.getLayoutParams();  
        params.gravity = Gravity.END;  
        params.leftMargin = dpToPx(64);  
        params.rightMargin = dpToPx(8);  
        holder.cardBubble.setLayoutParams(params);  
    } else {  
        // Réponse IA — gauche, bulle grise  
        holder.cardBubble.setCardBackgroundColor(0xFFFFF5F5F5);  
        LayoutParams params = (LayoutParams) holder.cardBubble.getLayoutParams();  
        params.gravity = Gravity.START;  
        params.leftMargin = dpToPx(8);  
        params.rightMargin = dpToPx(64);  
        holder.cardBubble.setLayoutParams(params);  
    }  
}
```

Annexe C — Extrait Backend FastAPI (main.py)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import httpx

app = FastAPI(title='QuizAppGuerouani API')
app.add_middleware(CORSMiddleware, allow_origins=['*'],
                  allow_methods=['*'], allow_headers=['*'])

class ChatRequest(BaseModel):
    message: str
    history: list = []

@app.post('/api/chat')
async def chat(req: ChatRequest):
    async with httpx.AsyncClient(timeout=120.0) as client:
        payload = {
            'model': 'llama3.2',
            'messages': req.history + [{'role': 'user', 'content': req.message}],
            'stream': False
        }
        r = await client.post('http://localhost:11434/api/chat', json=payload)
    return {'response': r.json()['message']['content']}
```